Master Build Prompt — AI-Powered Parental Task Delegation App

Target tool: Antigravity **Scope:** Phase 1 (Core Loop) + Phase 2 (Safety & Trust Hardening)
Stack: Android (Java) frontend · Python backend · OpenAI Whisper + GPT for STT/intent
Author's known languages: Java, Python — generate idiomatic, well-commented code in both.

---

0. Instructions to Antigravity (read first)

You are building a two-part system:

1. **Android app (Java)** — installed on the **parent's** phone only.
2. **Python backend service** — handles transcription, intent understanding, task preparation, and emergency detection.

Ground rules for this build:

- **No real-world irreversible actions.** Food ordering and cab booking are **simulated/mocked** in this scope. Do not integrate real Zomato/Swiggy/Uber/Ola APIs — generate a `MockExecutionService` that returns a fake confirmation after a short delay, as if a real order/booking succeeded.
- **Every non-emergency task requires explicit parent tap-to-approve before "execution."** Nothing fires automatically.
- **Emergency events bypass approval entirely** — see Section 6.
- Favor **clarity and modularity** over cleverness. Comment generously — this is a personal project the author will maintain and extend themselves.
- Where a real capability is legally/technically out of reach in a generated prototype (e.g., silently recording live phone calls on modern Android), **flag it clearly in code comments and in a `KNOWN_LIMITATIONS.md`** rather than faking a workaround.
- Do not fabricate working API keys, secrets, or credentials. Use `.env` / `local.properties` placeholders (`OPENAI_API_KEY=your_key_here`) and `.gitignore` them.

---

1. Call Capture (author's decision: automatic, on-device, triggered by registered number)

Confirmed design: When a call comes in from a phone number matching a registered

`ChildProfile.phone_number`, the app automatically captures the call audio on the parent's device — no manual step from the parent. The audio file is then auto-uploaded to the backend for Whisper transcription (Section 5). Nothing is transcribed on-device.

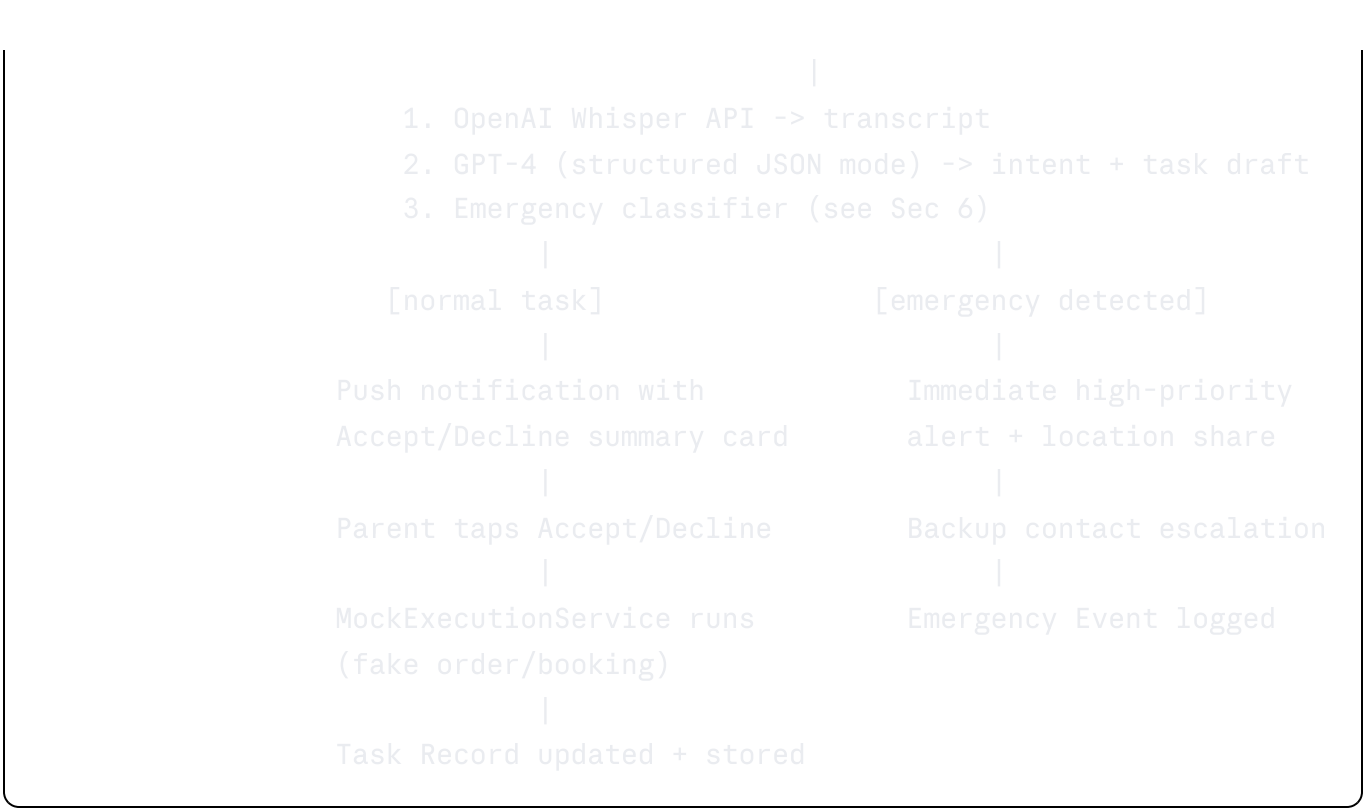
Build it this way, but be honest about the platform ceiling:

- Antigravity should build the **call-detection layer using `PhoneStateListener` / `TelephonyManager` (or `CallScreeningService` on Android 10+)** to detect call start/end, duration, and match the incoming number against `ChildProfile.phone_number`.
- **Automatic call *audio* capture is genuinely restricted on modern Android** (10+) unless the app holds the **default Dialer role**, and even then behavior is inconsistent across OEMs (Samsung, Xiaomi, Pixel all differ) and is disallowed in some regions/carriers entirely. Build `NativeCallRecordingSource` as the primary implementation, requesting the default-dialer role during onboarding and using `MediaRecorder`/`CallRecordingService` where the OEM permits it.
- **Build a `ManualUploadAudioSource` as an automatic fallback**, not the primary path: if `NativeCallRecordingSource` fails to produce a file after a call ends (permission denied, OEM blocks it, etc.), surface a lightweight in-app prompt asking the parent to share a call-recording file from their device's native recorder — this keeps the pipeline unblocked on devices where silent capture doesn't work, without changing the intended "automatic" UX on devices where it does.
- Add a **"How call capture works on your device" info screen** during onboarding, and a one-time compatibility check that tells the parent up front whether their specific phone/OEM supports automatic capture, so expectations are set correctly before they rely on it. This remains the single riskiest technical assumption in the whole build — test it early on Adi's actual device/carrier before building anything downstream of it.

---

2. High-Level Architecture

```
[Child's phone] --(normal phone call)--> [Parent's Android phone]
                                                  |
                                  (registered number matched, call
                                   auto-captured, call ends)
                                                  |
                                  Android app auto-uploads audio
                                  (falls back to manual share prompt
                                   only if native capture fails)
                                                  v
                                  [Python backend — FastAPI service]
```



---

3. Tech Stack (explicit)

Layer	Choice	Notes
Mobile app	Android, Java, min SDK 26 (Android 8.0+)	Use AndroidX, ViewBinding, MVVM pattern
Backend	Python, **FastAPI**	Async, auto-generated OpenAPI docs, easy to extend
STT	**OpenAI Whisper API** (`whisper-1`)	Send uploaded audio file, receive transcript
Intent/decision engine	**OpenAI GPT-4** with JSON-mode structured output	Prompted to output a fixed schema (Section 5)
Database	**SQLite** for local prototype, structured so it can migrate to **PostgreSQL** later	Use SQLAlchemy ORM so the swap is a config change
Notifications	**Firebase Cloud Messaging (FCM)**	Push summary cards + emergency alerts to parent's phone

Layer	Choice	Notes
Voice biometrics (Phase 2)	**Resemblyzer** (open-source speaker-embedding library) run inside the Python backend	Simple cosine-similarity match against an enrolled voice sample; good enough for a v1, documented as "basic" not "forensic-grade"
Location (Phase 2 emergency)	Android **FusedLocationProviderClient**	Only triggered on emergency bypass, never routine tasks
Auth	Firebase Authentication (email/phone)	Parent account login

4. Android App Structure (Java)

```
app/
  src/main/java/com/<pkg>/parentaltaskapp/
    ui/
      home/HomeActivity.java          // list of child profiles
      home/AddEditChildActivity.java  // add/edit a child profile
      history/HistoryActivity.java    // list of past Task Records
      history/TaskDetailActivity.java // transcript + summary + status
      approval/ApprovalCardActivity.java // Accept/Decline UI (launched from
push notification)
      emergency/EmergencyAlertActivity.java // full-screen, non-dismissable-
until-acknowledged
      profile/ProfileActivity.java    // parent account settings
      onboarding/CallCaptureInfoActivity.java // explains call-capture
limitations (Sec 1)
    data/
      model/ChildProfile.java
      model/TaskRecord.java
      model/EmergencyEvent.java
      repository/TaskRepository.java  // talks to backend via Retrofit
      repository/ChildRepository.java
    network/
      ApiService.java                 // Retrofit interface matching Sec 5
endpoints
      RetrofitClient.java
    call/
      CallAudioSource.java            // interface, see Sec 1
      ManualUploadAudioSource.java
      NativeCallRecordingSource.java  // stubbed, TODO
```

```
      CallStateReceiver.java          // detects call start/end via
TelephonyManager
    notifications/
      FcmService.java                 // handles incoming push, builds
approval/emergency notifications
    util/
      PermissionsHelper.java
      Constants.java
```

Key UX requirement (from masterplan Section 6): the **Approval Card** and **Emergency Alert** must be visually and behaviorally distinct — emergency uses a red, full-screen, sound+vibration alert with **no tap-to-proceed gate** (it's informational + action buttons like "Call child now" / "I'm on it", not "Accept this action").

---

5. Backend API Contract (FastAPI)

Generate these endpoints with Pydantic request/response models:

```
POST /calls/upload
  Body: multipart audio file + child_id + call_timestamp
  -> { call_id }

POST /calls/{call_id}/process
  Triggers: Whisper transcription -> GPT intent extraction -> emergency check
  -> {
       call_id, transcript, is_emergency: bool,
       task: {
         category: "food" | "cab" | "none",
         summary_text: str,          # plain-language, shown on approval card
         details: { ... },           # e.g. restaurant, items, cost, address
         confidence: float
       } | null,
       emergency_event: { trigger_reason, timestamp } | null
     }

POST /tasks/{task_id}/decision
  Body: { decision: "approved" | "declined" }
  -> triggers MockExecutionService if approved
  -> { status: "executed" | "declined" | "failed" }

GET /children/{child_id}/tasks
  -> list of Task Records (for History screen)

POST /children
  Body: { name, phone_number, age, backup_contact }
  -> creates ChildProfile, default approval_strictness derived from age

POST /children/{child_id}/enroll-voice
  Body: audio sample
  -> stores voice embedding (Resemblyzer) for Phase 2 verification

POST /emergency/{event_id}/acknowledge
  -> stops escalation timer to backup contact
```

GPT intent-extraction prompt (embed this in the backend, adjustable):

> "You are a task-intent classifier for a child's phone call to their parent. Given this transcript, output STRICT JSON only: `category` (food/cab/none), `summary_text` (one plain sentence a busy parent can read in 2 seconds), `details` (object), `confidence` (0-1), `is_emergency` (bool), `emergency_reason` (string or null). Treat distress, fear, pain,

> or safety threats as emergencies — bias toward false positives over false negatives for emergency detection."

---

6. Emergency Detection & Bypass (Phase 2 — critical path)

- Emergency classification happens **inside the same GPT call** as intent extraction (see prompt above) — keep it a single call to avoid latency doubling, but log both signals separately.
- On `is_emergency: true`:
  1. Skip the approval pipeline entirely.
  2. Fire an FCM high-priority notification that opens `EmergencyAlertActivity` even over lock screen.
  3. Trigger `FusedLocationProviderClient` to grab the child's phone's last known location **if the child's device also has a lightweight companion component** — flag clearly that this requires the child to have *some* app or SMS-based location share; a completely offline keypad phone cannot provide GPS. Document this gap explicitly.
  4. Start a backend timer (e.g., 3 minutes); if the parent hasn't acknowledged via `/emergency/{id}/acknowledge`, send a second **push notification (FCM)** to the backup contact. **No Twilio/SMS for now** — this means the backup contact must also have the app installed with a registered device token (they can be added as a lightweight "guardian" profile with no children of their own, just a notification target). Flag this as a real limitation: an offline/non-app backup contact cannot be reached in this version. Twilio SMS escalation is a clean Phase 3 add — build the escalation logic behind a `NotificationChannel` interface so swapping in SMS later doesn't require touching the emergency-detection logic.
- Because false positives here are costly (crying wolf) and false negatives are dangerous, build a simple **admin review flag**: every emergency event is stored and reviewable in a `History` filter, per masterplan's "log and review to refine" mitigation.

6a. Approval-Strictness Rule Engine (Phase 2, confirmed logic)

Two modes for now (design the enum so a third tier can be added later without a schema change):

- **`always_approve`** — every non-emergency task sits as `pending` until the parent taps Accept/Decline. No timeout auto-action. This is the default for younger children.

- **`auto_approve_under_threshold`** — the parent sets `auto_approve_max_amount` (e.g. ₹300) and `auto_approve_timeout_minutes` (parent-configurable, e.g. 2 or 5). When a task's estimated cost (from GPT's `details.cost`) is at or under the threshold:
  - The approval card still fires immediately, same as always.
  - A backend job starts a `auto_approve_timeout_minutes` timer alongside it.
  - If the parent hasn't responded when the timer fires, the backend auto-approves the task, sets `decision_source = "auto_timeout"`, and runs `MockExecutionService`. If cost is *above* the threshold, no auto-approve timer runs regardless of mode — it behaves like `always_approve` for that task.
  - Any parent response (Accept/Decline) before the timer fires cancels it and sets `decision_source = "manual"`.
- History screen should visually mark auto-approved tasks differently (e.g., a small "auto-approved" tag) so the parent can spot them at a glance and adjust settings if they're unhappy with what got through.

---

7. Data Model (generate as both SQLAlchemy models and Java POJOs)

```
Parent: id, name, email, auth_id, created_at
ChildProfile: id, parent_id, name, phone_number, age,
              approval_mode (enum: "always_approve" |
"auto_approve_under_threshold",
                             default "always_approve" for younger ages),
              auto_approve_max_amount (nullable, only used when approval_mode
is
                             auto_approve_under_threshold — e.g. tasks under
₹300),
              auto_approve_timeout_minutes (nullable, parent-configurable,
default 5 —
                             how long to wait with no response before auto-
approving),
              backup_contact_phone, voice_embedding (nullable, Phase 2)
TaskRecord: id, child_id, call_id, transcript, summary_text, category,
            details_json, status (pending/approved/declined/executed/failed),
            decision_source (enum: "manual" | "auto_timeout"),
            created_at, decided_at
EmergencyEvent: id, child_id, trigger_reason, transcript_snippet,
                location_lat, location_lng (nullable),
backup_contact_notified (bool),
                acknowledged (bool), created_at
```

No raw audio retained beyond active processing — delete the uploaded file after transcription completes (masterplan Section 7).

---

8. Security Requirements (Phase 1 + 2)

- Caller verification, two layers: (1) incoming number matches `ChildProfile.phone_number`; (2) Resemblyzer voice-embedding cosine similarity above a threshold (Phase 2).
- All backend endpoints require a Firebase Auth JWT tied to the parent account; a child profile can only be accessed by its owning parent.
- Encrypt `voice_embedding` and any stored transcripts at rest (SQLCipher for SQLite, or Postgres native encryption later).
- Rate-limit `/calls/upload` per child to prevent abuse.
- All secrets via environment variables — never hardcoded.

---

9. Build Order for Antigravity

1. Scaffold Python FastAPI backend with SQLite + SQLAlchemy models (Sec 7), stub endpoints (Sec 5) returning mock data.
2. Wire real Whisper + GPT calls into `/calls/{id}/process`.
3. Build `MockExecutionService` for food/cab.
4. Scaffold Android app: Home/History/Profile screens, Retrofit client against the backend.
5. Build `ManualUploadAudioSource` flow (parent shares a recording into the app) end-to-end through to an Approval Card.
6. Add FCM push notifications for approval cards.
7. Add Phase 2: voice enrollment + Resemblyzer verification, emergency detection + `EmergencyAlertActivity`, backup-contact escalation via a second FCM push (backup contact registered as a guardian profile), the auto-approve timeout job (Section 6a), and per-child approval-strictness settings UI (mode toggle + threshold amount + timeout minutes).
8. Write `KNOWN_LIMITATIONS.md` covering call-recording reliability across OEMs and the child-side location gap.

---

10. Explicitly Out of Scope (do not build)

- Real Zomato/Swiggy/Uber/Ola integrations (Phase 3, future work).
- iOS app.
- Forensic-grade voice biometrics.
- Any auto-execution without parent approval outside the emergency path.

---

11. Decisions Log

- **Call capture:** Automatic, on-device, triggered when a call comes in from a registered child number. Audio is auto-uploaded to the backend for Whisper transcription (not transcribed on-device). Manual upload is a fallback path only, used if native capture fails on a given OEM/device.
- **Emergency escalation:** Push notification only for now (no Twilio/SMS). The backup contact must have the app installed as a "guardian" profile to receive it. SMS escalation is deferred to Phase 3, built behind a `NotificationChannel` interface so it's a clean swap-in later.
- **Approval strictness:** Two concrete modes — `always_approve` and `auto_approve_under_threshold` (with a parent-set cost threshold and timeout, default 5 minutes). See Section 6a for full logic.

Still genuinely open — worth deciding before/while building

- Which device/carrier will you test native call-recording on first? This tells you early whether you'll be relying on the manual-upload fallback more than you'd like.
- Should the compatibility check (Section 1) run once at onboarding only, or re-check periodically in case an OS update changes what's allowed on that device?
- For a guardian-only backup contact profile, do they need their own login, or can they just register a device token via a lightweight invite link (no full account)?